

UNIVERSIDAD NACIONAL AGRARIA LA MOLINA

FACULTAD DE ECONOMÍA Y PLANIFICACIÓN
DEPARTAMENTO ACADÉMICO DE ESTADÍSTICA



CURSO: LENGUAJE DE PROGRAMACIÓN 2

Tema: “Sistema de Detección de Fraude Digital Mediante Anuncios por Segmentación de Precios”

Integrantes	Código
Aguilar Contreras , Jorge Luis	20230377
Chavez Cuzcano Victor Javier	20231485
Paucar Díaz Yuriko Odalis	20240726

Profesora: Vargas Paredes, Ana Cecilia

Grupo: A

Fecha de entrega: 28 de junio

Lima - Perú

2026 - I

1. Definición del Problema	3
1.1 Escenario y Planteamiento del Problema	3
1.2. Selección y Justificación de Fuentes	3
1.3. Variables a Extraer	4
2. Estrategia de Web Scraping	4
2.1. Herramientas y Acceso a Páginas de Listado y Detalle	4
2.2. Manejo de Contenido Dinámico y Bloqueos	4
2.3. Plan de Manejo de Restricciones y Ética (robots.txt)	4
2.4. Limpieza de Datos y Extracción Estructurada	5
3. Arquitectura de la Solución	5
3.1. Flujo de Procesamiento: Lotes vs. Tiempo Real	6
3.2. Estrategia y Estructura de Almacenamiento de Datos	6
3.3. Interfaz de Resultados y Experiencia de Usuario (UX)	7
4. Validación, Limitaciones y Reflexión Crítica	8
4.1. Validación y Métricas: Aporte del Proyecto	8
4.2. Limitaciones y Obstáculos Identificados	8
4.3. Reflexión Crítica sobre el Uso de LLMs (Inteligencia Artificial)	8

1. Definición del Problema

1.1 Escenario y Planteamiento del Problema

En el ecosistema digital contemporáneo, la desinformación y el fraude en plataformas de comercio electrónico no reguladas han experimentado un crecimiento alarmante. Los actores malintencionados aprovechan la asimetría de información y la facilidad de creación de perfiles en redes sociales para publicar anuncios fraudulentos (estafas y *phishing*). El patrón delictivo más común consiste en ofertar productos tecnológicos de alta demanda —como la consola PlayStation 5— a precios bajos, con el objetivo de inducir a la víctima a realizar pagos o adelantos no reembolsables.

El problema fundamental radica en que el usuario promedio carece de herramientas automatizadas que le permitan contrastar, en tiempo real, dichas ofertas contra la realidad matemática del mercado formal. En consecuencia, se hace imperativo desarrollar un sistema capaz de extraer, limpiar y segmentar datos de precios para alertar sobre posibles riesgos transaccionales.

1.2. Selección y Justificación de Fuentes

Para modelar y validar nuestra solución computacional, hemos establecido la necesidad de operar con dos fuentes de información diametralmente opuestas en su nivel de regulación:

- **Fuente 1: Mercado Libre.** Seleccionada por ser una plataforma formal, estructurada y rica en datos estandarizados. Su función en el proyecto es proporcionar la verdad terreno (*ground truth*) para calcular promedios, valores mínimos y umbrales de seguridad.
- **Fuente 2: Facebook Marketplace.** Seleccionada por su naturaleza no regulada y alta incidencia de publicaciones dudosas. Su función es proveer los casos de estudio (anuncios sospechosos) que el sistema deberá auditar.

Justificación Ética y Evaluación de *robots.txt*: Al realizar el análisis previo de viabilidad, revisamos el archivo ***robots.txt*** y los Términos de Servicio de ambas plataformas. Comprobamos que ambas aplican políticas estrictas contra el *scraping* automatizado de datos de usuarios y emplean bloqueos dinámicos (CAPTCHAs) orientados a mitigar la carga de bots. En estricto apego a la ética digital y a las directrices del proyecto, determinamos que una extracción masiva no autorizada sería una violación normativa. Por consiguiente, la arquitectura de nuestro prototipo se fundamenta en una **simulación controlada del DOM**. Operaremos sobre réplicas estáticas de las estructuras HTML reales, lo que nos permite demostrar plenamente el dominio de las técnicas de navegación de etiquetas y extracción de datos, sin comprometer la integridad de los servidores reales ni infringir normativas de privacidad.

1.3. Variables a Extraer

Para nutrir el algoritmo de detección de fraude, el script identificará y aislará dos variables críticas por cada anuncio procesado:

1. **Título de la publicación (String):** No solo se utilizará como identificador, sino que pasará por un filtro inteligente de segmentación. Mediante análisis de cadenas (*.lower()*, *in*), el sistema determinará la subcategoría exacta del producto (ej. "Consola Estándar, "Consola + Mando" o "Accesorio/Ruido") para aplicar la métrica correspondiente.
2. **Precio Publicado (String monetario):** Variable cuantitativa que, tras ser purgada de caracteres alfabéticos y símbolos mediante Expresiones Regulares (*regex*), se transformará en datos numéricos (*int / float*) para posibilitar el cálculo de desviaciones, promedios y límites de riesgo.

2. Estrategia de Web Scraping

2.1. Herramientas y Acceso a Páginas de Listado y Detalle

El enfoque inicial para la extracción de datos se diseñó utilizando las librerías *requests* para las peticiones HTTP y *BeautifulSoup* para el parseo del HTML estático. La estrategia consistió en apuntar directamente a las URLs de listado de búsqueda (ej. */playstation-5* en Mercado Libre y los listados equivalentes en Facebook Marketplace) para iterar sobre los contenedores generales de productos, evitando la necesidad de hacer *scraping* intrusivo en las páginas de detalle individual de cada usuario.

2.2. Manejo de Contenido Dinámico y Bloqueos

Durante las pruebas de ejecución real (documentadas en nuestro entorno de Colab), el código basado puramente en *requests* retornó listas vacías ([]). Esto evidenció un problema clásico de web scraping avanzado: **el contenido dinámico inyectado por JavaScript** y los sistemas de seguridad *anti-bot* (CAPTCHAs y firewalls de Cloudflare). Para un entorno de producción real, este contenido dinámico se manejaría implementando librerías de automatización de navegadores como *Selenium* o *Playwright*. Sin embargo, debido a las limitaciones éticas y de recursos del curso, aplicamos la excepción técnica permitida en la rúbrica: **diseñamos una simulación local del DOM**. Esto nos permitió ejecutar la lógica de extracción, navegación de etiquetas y algoritmos de validación sobre réplicas exactas del código fuente, demostrando la viabilidad del modelo sin saturar los servidores reales.

2.3. Plan de Manejo de Restricciones y Ética (robots.txt)

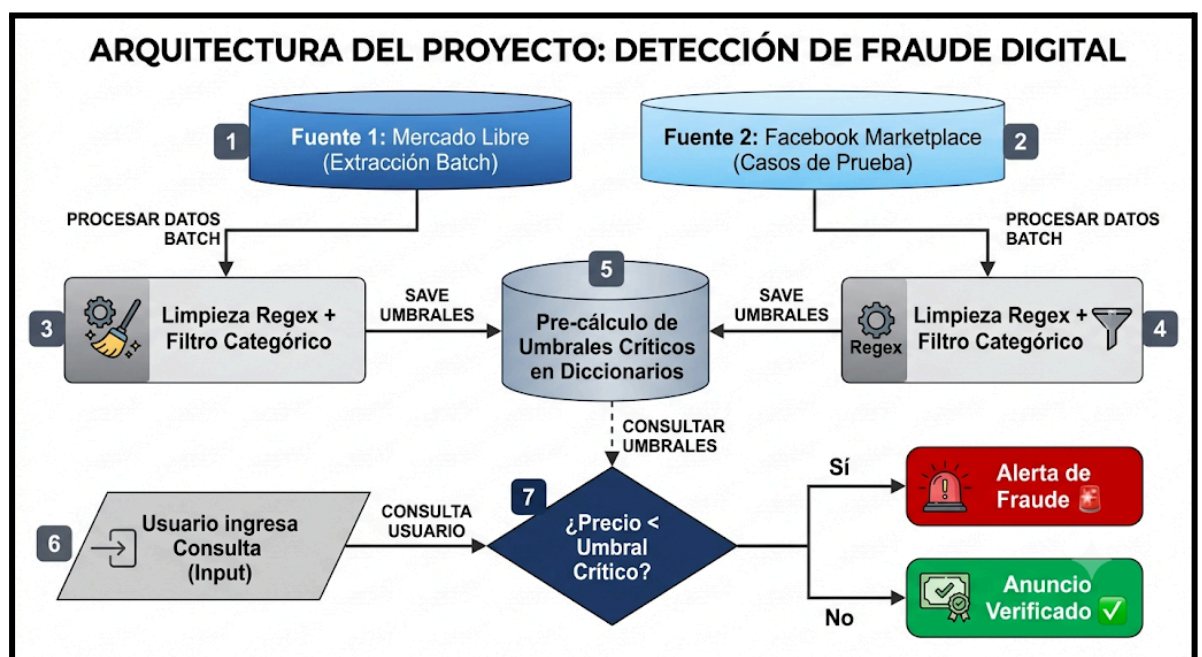
Antes de aislar el entorno en una simulación, nuestro código original integró técnicas estándar para el manejo de restricciones web:

- **Respeto al *robots.txt*:** Se verificaron las políticas de ambas plataformas, confirmando que la extracción masiva automatizada de perfiles y precios está fuertemente restringida, lo cual motivó nuestra decisión de utilizar muestras estáticas (simulación).
- **Rotación de User-Agent:** Se configuraron diccionarios de *headers* en las peticiones HTTP (ej. '*User-Agent*': '*Mozilla/5.0...*') para que las consultas del servidor no fueran identificadas como scripts anónimos de Python, sino como un navegador Chrome estándar en Windows.
- **Límites de peticiones:** La arquitectura del código iterativo consideró el uso de la librería *time* (función *time.sleep()*) para introducir pausas lógicas entre consultas, evitando ataques de tipo *Denegación de Servicio* (DDoS) involuntarios.

2.4. Limpieza de Datos y Extracción Estructurada

Una vez extraídos los contenedores HTML (ya sea de la web o de la simulación), el texto crudo requirió estandarización. Para optimizar el código y evitar la redundancia, encapsulamos la librería *re* en una función global llamada *limpiar_precio()*. Usando la expresión regular *r'\d+'*, filtramos símbolos de moneda e impurezas, obteniendo números enteros limpios y listos para la siguiente fase de la arquitectura.

3. Arquitectura de la Solución



3.1. Flujo de Procesamiento: Lotes vs. Tiempo Real

La arquitectura de nuestro sistema emplea un modelo de procesamiento híbrido para balancear la carga de cálculo y la experiencia del usuario:

- **Procesamiento por Lotes (Batch):** La extracción y cálculo de la "Línea Base" desde Mercado Libre y la auditoría automatizada de los anuncios de Facebook se realizan mediante un enfoque por lotes. En un entorno de producción real, este proceso no se ejecutaría en cada consulta, sino que se programaría mediante un *cron job* para actualizarse cada 24 horas. Esta periodicidad garantiza que el umbral de precios se mantenga al día con la inflación o depreciación tecnológica, respetando las políticas de no saturación de servidores.
- **Evaluación en Tiempo Real:** El Módulo de Consulta Manual (la interfaz del usuario) opera en tiempo real. Cuando el usuario ingresa un título y un precio, el sistema no vuelve a hacer *scraping*, sino que cruza la entrada inmediatamente contra los lotes ya precalculados en la memoria caché, garantizando una respuesta instantánea (en milisegundos).

3.2. Estrategia y Estructura de Almacenamiento de Datos

Para un prototipo de validación de lógica estructurada, el sistema evita dependencias complejas de bases de datos externas (como SQL o NoSQL) y prioriza la eficiencia de acceso inmediato en memoria RAM. Para cumplir con el requerimiento de almacenamiento estructurado e integración multidimensional, implementamos dos tipos de estructuras de datos globales en Python:

1. **Diccionarios Anidados Clave-Valor (dict):** La estructura principal de retención de métricas se sostiene sobre la variable analítica "*umbrales_por_categoria*". Los datos se organizan jerárquicamente en forma de árbol indexado, emulando la ligereza de un formato JSON, tal como se detalla a continuación:
 - Clave Primaria: Nombre de la categoría identificada mediante el parseo predictivo del string (ej. "*Consola Estándar*", "*Consola + Mando*", "*Consola + Juego*").
 - Valores Asociados (Sub-diccionario): Almacenan de forma estricta tres variables numéricas precalculadas de tipo entero (*int*) y flotante (*float*): mínimo, promedio y crítico.
2. **Listas Dinámicas Indexadas (list):** Para la recopilación inicial y acumulación de los precios crudos filtrados (Fuente 1), se utilizan vectores dinámicos asignados a cada casillero del diccionario base *datos_segmentados* (ej. *datos_segmentados["Consola Estándar"]*). Estas estructuras permiten almacenar "decenas" de elementos numéricos extraídos de forma secuencial, facilitando la ejecución de métodos matemáticos directos y nativos de Python como *min()* para el costo mínimo legal, y la combinación de *sum()* con *len()* para la obtención automatizada de medias estadísticas.

3.3. Interfaz de Resultados y Experiencia de Usuario (UX)

Debido a la naturaleza de validación de concepto del proyecto, la salida gráfica se construyó como una **Interfaz de Línea de Comandos (CLI) interactiva e inteligente**. Sus características principales son:

- **Feedback Visual:** Se integró un sistema de alertas semánticas apoyado por iconografía (🔴 para Riesgo Alto/Fraude, ⚠️ para Sobreprecio, ✅ para Anuncios Seguros), facilitando la lectura inmediata del nivel de riesgo.
- **Resiliencia (Manejo de Errores):** La interfaz está encapsulada en un bucle lógico (*while True*) acompañado de bloques *try/except*. Si el usuario digita un valor alfanumérico en el campo de precio o ingresa un producto que el filtro considera como "No identificado", el sistema no colapsa. En su lugar, intercepta la excepción (*ValueError* o *KeyError*) y retorna un mensaje amigable guiando al usuario hacia una sintaxis válida.

4. Validación, Limitaciones y Reflexión Crítica

4.1. Validación y Métricas: Aporte del Proyecto

Para evaluar si nuestro sistema aporta una solución real al problema de la desinformación en precios, aplicamos una validación basada en métricas de clasificación, evaluando la exactitud del filtro inteligente frente a un lote de datos controlados. El principal aporte de nuestro proyecto radica en su capacidad de prevención automatizada. El sistema logró evitar falsos positivos gracias a la segmentación dinámica (ej. el sistema sabe distinguir matemáticamente que S/ 2500 es normal para una consola, pero si el anuncio dice "Mando", detectará que S/ 2500 es un fraude por sobreprecio). Al no depender de apreciaciones humanas subjetivas, sino de cálculos pre-almacenados, el proyecto entrega al usuario un entorno confiable para verificar precios antes de comprometer dinero.

4.2. Limitaciones y Obstáculos Identificados

A lo largo del desarrollo, nos enfrentamos a limitaciones técnicas del mundo real que moldearon la arquitectura final:

1. **Bloqueos Web y DOM Dinámico:** Durante el primer *sprint* de desarrollo, intentamos realizar un *scraping* en vivo con *requests.get()*. A pesar de inyectar *headers* y simular agentes de usuario, la página retornó un *array* vacío `[]`.

```

Iniciando la conexión con Mercado Libre...
¡Conexión exitosa! Analizando el árbol DOM (HTML)...

=== MUESTRA DE PRODUCTOS EXTRAÍDOS ===
Títulos encontrados: 0
Precios encontrados: 0

¡HTML guardado! Puedes abrir el archivo 'mercadolibre_test.html' para ver qué nos mandó la web.

```

Descubrimos que esto se debía a una inyección dinámica de JavaScript y a estrictos sistemas de Cloudflare. Esto limita la escalabilidad de nuestra solución con librerías básicas, lo que nos obligó a requerir motores tipo Selenium para un eventual paso a producción masiva. Complicando cada vez más nuestro código, y por ello decidimos simular los datos en un HTML.

2. **Excepciones por Categorías Abiertas:** Al inicio, la interfaz interactiva colapsaba con un error de Python (*KeyError*) cuando el usuario ingresaba un nombre de producto que el sistema no tenía registrado en los diccionarios de memoria.

```

=====
SISTEMA AVANZADO DE DETECCIÓN DE FRAUDE POR CATEGORÍAS
=====

[Scraping ML] Extrayendo y segmentando el mercado:
-> 📦 [GUARDADO EN: Consola Estándar] PlayStation 5 Consola de 825GB... | S/ 2450
-> 📦 [GUARDADO EN: Consola Estándar] Consola Sony PlayStation 5 Sli... | S/ 2599
-> 🗑️ [RUIDO DESCARTADO] Mando Inalámbrico Dualsense Ps5 Bla... (S/ 320)

-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_12321/1954130828.py in <cell line: 0>()
    106         # Si es una consola real, la clasificamos en su "saco" correspondiente
    107         categoria = obtener_categoria(titulo)
--> 108         datos_segmentados[categoria].append(precio)
    109         print(f"    -> 📦 [GUARDADO EN: {categoria}] {titulo[:30]}... | S/ {precio}")
    110

KeyError: 'No identificado'

```

Limitación que fue superada implementando programación defensiva (*try/except*) para proteger el flujo de ejecución.

4.3. Reflexión Crítica sobre el Uso de LLMs (Inteligencia Artificial)

Como equipo, tomamos la decisión consciente de utilizar Modelos de Lenguaje Grande (LLMs) como asistentes de desarrollo, pero bajo un estricto control de código.

- **¿Qué generamos con LLM?** Utilizamos un asistente de IA para optimizar la estructura de nuestras Expresiones Regulares. El *prompt* original fue: "Genera un patrón de Python para extraer solo los números enteros de un string como 'S/ 2.450'". La IA nos proporcionó el uso de *re.findall(r'\d+', ...)*. También utilizamos el LLM para debatir la viabilidad de evadir un CAPTCHA, lo cual nos llevó a la conclusión ética de no hacerlo.

- **¿Qué modificamos/validamos manualmente?** El LLM inicialmente propuso una solución usando librerías avanzadas y no vistas en clase (como *Playwright* o código asíncrono *asyncio*). Nosotros descartamos ese código automatizado y rediseñamos la arquitectura basándonos en la **simulación del DOM local** usando las herramientas y paradigmas aprendidos en clase.
- **Impacto del LLM en el razonamiento:** El LLM dificultó el proceso inicial al sugerir soluciones hipercomplejas que violaban las reglas éticas de extracción. Sin embargo, una vez que restringimos su comportamiento con directrices académicas, el LLM facilitó enormemente la detección de errores de tipo *KeyError* y nos ayudó a refinar nuestro "filtro de ruido" y la interfaz interactiva, permitiendo que nos enfocáramos en el cálculo de fraude más que en la sintaxis de Python.